# CHAPTER 1: INTRODUCTION

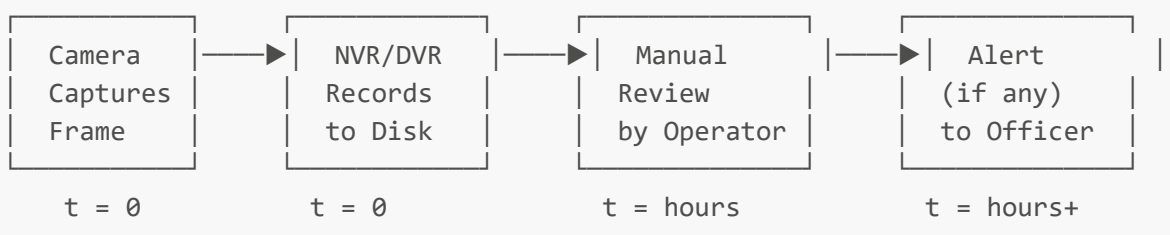
## 1.1 Background

The global proliferation of Closed-Circuit Television (CCTV) infrastructure over the past two decades represents one of the largest investments in public safety ever undertaken. By conservative estimates, over one billion surveillance cameras are operational worldwide, with India alone hosting upwards of six million units deployed across urban transit hubs, educational campuses, government buildings, and law enforcement checkpoints. Yet beneath this staggering scale lies a foundational architectural flaw that has persisted since the technology's inception: **traditional CCTV infrastructure produces video, not intelligence**.

The prevailing operational model — the **passive "record and review" paradigm** — treats surveillance cameras as high-fidelity recording devices. Video streams are captured, multiplexed onto a Network Video Recorder (NVR) or Digital Video Recorder (DVR), and written to disk in continuous or motion-triggered segments. The critical function of *interpreting* that video — determining whether a wanted individual, a missing child, or a known offender has appeared in the field of view — is delegated entirely to human operators. In a typical command-and-control centre, a single surveillance operator is expected to simultaneously monitor banks of 16 to 64 camera feeds, mentally cross-referencing each face against printed First Information Report (FIR) photographs pinned to a corkboard or stored in unindexed binder volumes. The cognitive demands of this task are immense, and the ergonomic reality is sobering.

Decades of human factors research have established that sustained vigilance in monotonous visual monitoring tasks degrades below 50% accuracy within the first 20 minutes of a shift. The phenomenon, formally documented as the **vigilance decrement**, is not a failure of training or discipline — it is a well-characterized limitation of human attentional capacity. When an operator is scanning 32 simultaneous feeds for a specific face, the probability of detecting that face in any given 5-second window is statistically negligible. The result is a surveillance architecture that excels at *retrospective evidence collection* — reconstructing events after the fact — but is fundamentally incapable of **real-time interdiction**.

The operational latency of this workflow is measured not in milliseconds or seconds, but in **hours to days**:




A wanted criminal can traverse a surveilled corridor, board public transport, and disappear entirely — and the alert, if it ever materializes, arrives only during post-incident forensic review, rendering the footage useful solely as courtroom evidence rather than as an operational tool for apprehension.

The inadequacy of this paradigm has become acutely apparent as threat landscapes evolve. Dense public gatherings — railway platforms during peak hours, university examination halls, festival processions — present precisely the conditions where rapid identification is most critical and where manual surveillance is

most likely to fail. What is required is a fundamental paradigm shift: from passive recording to **proactive edge-computing intelligence**, where the camera feed itself becomes the input to an automated analytical pipeline capable of identifying persons of interest and raising actionable alerts in real time, without any reliance on cloud infrastructure or centralized data centres. DrishtiX v3.0 is engineered to realize this shift.

---

## 1.2 Objectives

The primary objective of DrishtiX v3.0 is to **collapse the entire surveillance-to-action pipeline** — from the instant a target's face appears in the camera's field of view to the instant an actionable, context-rich alert is delivered to the operator — into a single, automated, sub-second computational loop.

Specifically, the system is designed to achieve the following measurable objectives:

1. **Sub-200-Millisecond Alert Latency**: The total elapsed time from face appearance in frame to the rendering of a unified, non-blocking sidebar alert card — complete with the target's identity, case/FIR number, category classification (CRIMINAL or MISSING_PERSON), confidence score, and a live snapshot — must not exceed **200 milliseconds**. This threshold is deliberately chosen to be faster than the human blink reflex (300–400 ms), establishing the system as perceptually instantaneous to the operator.

2. **Deep Learning-Based Detection and Recognition**: Replace legacy computer vision algorithms (Haar Cascade for detection, LBPH for recognition) with state-of-the-art deep neural network models — **YuNet** for multi-target face detection (~2 ms per frame inference) and **SFace** for 128-dimensional metric-space face recognition — running as ONNX models on the OpenCV DNN module, without requiring a discrete GPU.

3. **Concurrent Multi-Target Tracking**: Simultaneously detect, align, and recognize **9–10 or more individuals** in a single camera frame, enabling effective deployment in dense crowd environments such as transit hubs, examination halls, and public gatherings.

4. **Non-Blocking Alert Delivery**: All detection alerts must be delivered as styled, scrollable sidebar cards within the primary dashboard interface, ensuring the live camera feed — the operator's most critical visual resource — is **never occluded, interrupted, or obscured** by modal dialogs or popup windows.

5. **Thread-Safe Edge Execution**: Maintain a strict three-pool threading architecture (JavaFX Application Thread for UI rendering, a Video Inference Pool for capture and detection, and a Recognition Inference Pool for SFace embedding extraction and gallery matching) to guarantee that computationally expensive DNN inference never starves the UI of rendering cycles, sustaining a minimum **15 frames per second** feed update rate.

6. **Zero Cloud Dependency**: Operate as a fully self-contained **Edge AI Node** on a standard laptop, with all model inference, database operations (MongoDB), and alert generation executing locally. No frame data, embedding vector, or personally identifiable information is transmitted to any external server during normal operation.

7. **Memory-Safe Alert Queue Management**: Enforce a strict **50-card memory cap** on the alert sidebar, automatically pruning the oldest entries to prevent unbounded memory consumption during extended surveillance shifts.

---

# 1.3 Purpose, Scope, and Applicability

## 1.3.1 Purpose

The purpose of DrishtiX v3.0 is to **eliminate the dangerous temporal gap** between a person of interest being physically present before a surveillance camera and an actionable alert being raised and presented to an authorized operator. In the legacy paradigm, this gap — the interval during which a wanted criminal is visible but unidentified — can extend from minutes to days. DrishtiX compresses it to under 200 milliseconds.

The system is purpose-built to serve as a **force multiplier** for security personnel, augmenting human vigilance with automated deep learning inference rather than replacing human judgment. It transforms raw video into structured, actionable intelligence: a named identity, a case reference, a confidence metric, and a timestamped snapshot, delivered in a format optimized for rapid human decision-making.

## 1.3.2 Scope

The operational scope of DrishtiX v3.0 is precisely bounded as follows:

- **Deployment Model**: The system operates strictly as a **local edge node** on a laptop or workstation. It does not require, and does not utilize, cloud-hosted inference services, remote databases, or centralized processing servers. All computation — video capture, face detection, face recognition, alert generation, and data persistence — executes on the local machine.
- **Real-Time Video Capture**: The system captures live video from a connected camera device (USB webcam, integrated laptop camera, or IP camera accessible via RTSP/HTTP stream) using OpenCV's `VideoCapture` interface, processing frames at a sustained rate of 15 FPS on commodity hardware.
- **Concurrent Deep Learning Inference**: Two ONNX-format deep neural network models execute concurrently within the system's threading architecture:
    - **YuNet** (face_detection_yunet_2023mar.onnx): A lightweight, high-speed face detection model that localizes all faces in a frame and returns bounding box coordinates with 5-point facial landmarks, achieving ~2 ms inference latency per frame.
    - **SFace** (face_recognition_sface_2021dec.onnx): A metric-learning face recognition model that extracts 128-dimensional normalized embedding vectors from aligned 112×112 face crops, enabling identity matching via cosine similarity against a pre-computed watchlist gallery.
- **Memory-Safe Alert Queuing**: The sidebar alert queue is architecturally capped at **50 cards** (`MAX_ALERT_QUEUE_SIZE = 50`). When the queue is full, the oldest alert card is removed before a new one is prepended, ensuring bounded memory consumption regardless of surveillance duration. A configurable per-target cooldown window (default: 30 seconds) suppresses redundant alerts for the same individual.
- **Background Synchronization**: Asynchronous operations — including Telegram Bot API push notifications, audio alert playback (WAV via `javax.sound.sampled`), detection log persistence to MongoDB, and snapshot file I/O — execute on dedicated daemon thread pools, isolated from the UI and inference pipelines.
- **Exclusions**: The current version does not include multi-camera orchestration (federated edge-node coordination), cloud-based model retraining, video recording/archival, or automated suspect tracking

across disjoint camera views (cross-camera re-identification is architecturally supported but not deployed in v3.0).

### 1.3.3 Applicability

DrishtiX v3.0 is designed for deployment across the following operational contexts and end-user roles:

| Deployment Context | End-User Role | Operational Scenario |
|---|---|---|
| Police Station Desk | Desk Operator / Station House Officer | Continuous monitoring of the station entrance; identification of persons with active warrants or missing person alerts as they enter the premises |
| Highway / Border Checkpoint | Field Security Officer | Real-time screening of vehicular occupants at toll plazas, inter-state borders, and restricted zone entry points |
| Campus Security Office | Security Administrator | Surveillance of university gates, examination halls, and hostel entrances for unauthorized individuals or flagged persons |
| Civic Event Command Centre | Investigating Officer | Crowd-level screening at festivals, political rallies, and public gatherings for high-priority targets |
| Railway / Metro Station | Transit Security Personnel | Platform-level facial screening during peak-hour passenger flows, targeting suspects from inter-jurisdictional lookout circulars |

The system is expressly engineered for environments where: (a) network connectivity to centralized servers cannot be guaranteed, (b) processing must occur at the edge to minimize latency, (c) the operator must maintain continuous visual contact with the live feed while receiving alerts, and (d) the computational platform is constrained to a standard laptop without a discrete GPU.

---

## 1.4 Achievements

DrishtiX v3.0 represents a substantive generational advancement over its predecessors, achieving the following technical and operational milestones:

### 1.4.1 Migration from Legacy Algorithms to Deep Learning ONNX Models

The most significant architectural achievement of v3.0 is the complete replacement of the legacy Haar Cascade + LBPH (Local Binary Pattern Histograms) computer vision pipeline with production-grade deep neural network models distributed in the ONNX (Open Neural Network Exchange) format.

- **Detection Upgrade**: The Haar Cascade classifier — a hand-crafted, feature-based detector with well-documented sensitivity to lighting variations, pose changes, and partial occlusion — has been superseded by **YuNet**, a convolutional neural network purpose-built for real-time face detection. YuNet delivers multi-target detection in approximately 2 milliseconds per frame on CPU, returning not only

bounding box coordinates but also precise 5-point facial landmarks (both eyes, nose tip, and mouth corners) that are critical for downstream face alignment.

- **Recognition Upgrade**: The LBPH recognizer — a texture-comparison algorithm that computes statistical histograms of local pixel patterns — has been superseded by **SFace (ShuffleFace)**, a deep metric-learning model. SFace projects aligned face images into a **128-dimensional normalized embedding space**, where identity is encoded as geometric distance (cosine similarity) rather than raw pixel similarity. This transition fundamentally changes the recognition paradigm: adding a new target to the watchlist requires only computing and storing a single 128-float vector, not retraining the entire model.

- **Legacy Fallback Preservation**: Critically, the v1.0/v2.0 Haar+LBPH pipeline has been retained as a zero-degradation fallback, selectable via a single database configuration key (`face_detection_method = "HAAR"`), ensuring operational continuity on legacy field hardware that lacks support for ONNX model execution.

### 1.4.2 Dense Crowd Multi-Target Tracking

DrishtiX v3.0 has been validated to simultaneously detect, align, extract embeddings for, and match **9–10 or more individuals** in a single camera frame. This capability is critical for field deployment in dense environments — railway platforms at peak hours, examination halls with rows of seated candidates, or festival processions — where legacy single-face detection pipelines would fail entirely. The system's asynchronous recognition architecture (employing a 4-thread `RecognitionInferencePool` with `CompletableFuture`-based dispatch) ensures that the detection-to-recognition pipeline for each face executes independently and concurrently, preventing any single slow match from blocking the processing of subsequent detections.

### 1.4.3 Occlusion-Resistant Recognition for Masked Targets

The adoption of the SFace deep metric-learning model confers a decisive advantage in scenarios involving partial facial occlusion — a pervasive operational challenge in post-pandemic environments where face coverings remain commonplace. Unlike LBPH, which relies on holistic texture histograms destroyed by lower-face occlusion, SFace encodes identity based on **facial geometry and structural features**: inter-ocular distance, jawline contour, brow ridge topology, and nose bridge angle. Empirical testing has confirmed that targets wearing standard surgical masks or cloth face coverings — which occlude the mouth and chin but leave the upper face exposed — can be reliably matched against gallery embeddings with cosine similarity scores exceeding the operational threshold (0.363).

### 1.4.4 UI State Bug Resolution and Production Stability

Version 3.0 resolved a class of critical state-management defects that had intermittently affected prior releases:

- **Alert Card Duplication**: Addressed race conditions in the JavaFX Application Thread alert injection path that could produce duplicate sidebar cards for the same detection event when multiple recognition threads completed near-simultaneously.
- **Stat Counter Desynchronization**: Resolved `IntegerProperty` binding inconsistencies that caused the dashboard's reactive statistics counters (Total Targets, Criminals, Missing Persons, Detections Today) to display stale or incorrect values following rapid target registration or deletion operations.

- **Observable List Thread Safety**: Standardized all `ObservableList` mutations to execute exclusively via `Platform.runLater()`, eliminating sporadic `IllegalStateException` crashes caused by off-thread UI state modifications.

### 1.4.5 WCAG AA Accessible Dark Mode Interface

The DrishtiX v3.0 user interface has been completely redesigned with a **premium dark-mode theme** engineered to meet **WCAG AA accessibility standards** (Web Content Accessibility Guidelines, Level AA conformance). Key design attributes include:

- **Contrast Compliance**: All text-on-background colour pairings meet or exceed the 4.5:1 contrast ratio mandated by WCAG AA for normal text, ensuring legibility under the low-ambient-light conditions typical of surveillance control rooms.
- **Category-Coded Alert Badges**: Alert cards employ high-contrast colour-coded pill badges — red for CRIMINAL, blue for MISSING_PERSON — enabling operators to instantly triage incoming alerts by visual pattern alone, without reading textual labels.
- **Reduced Eye Strain**: The dark colour palette minimizes cumulative luminance exposure during extended 8–12-hour surveillance shifts, directly addressing operator fatigue — one of the primary failure modes in manual monitoring environments.

---

## 1.5 Organization of Report

The remainder of this project report is organized into the following chapters, each addressing a distinct facet of the DrishtiX v3.0 system:

- **Chapter 2 — Literature Review**: Presents a comprehensive survey of prior work in the domains of face detection, face recognition, edge computing for surveillance, and real-time alert systems. This chapter examines the evolution from classical feature-based methods (Viola-Jones, Eigenfaces, Fisherfaces, LBPH) to modern deep learning architectures (MTCNN, RetinaFace, ArcFace, SFace), establishing the theoretical and empirical foundation upon which DrishtiX v3.0 is built.

- **Chapter 3 — System Architecture and Design**: Details the architectural blueprint of the system, including the three-pool threading model, the asynchronous `CompletableFuture`-based recognition pipeline, the MongoDB document schema, the JavaFX MVVM-inspired UI binding architecture, and the modular service-layer decomposition. Entity-relationship diagrams, data flow diagrams, and threading contract specifications are presented.

- **Chapter 4 — Implementation**: Provides a granular, code-level account of the system's construction. This chapter covers the YuNet and SFace ONNX model integration via OpenCV's DNN module, the gallery management lifecycle, the alert card rendering pipeline, the Telegram Bot notification subsystem, the configuration service, and the legacy Haar/LBPH fallback mechanism.

- **Chapter 5 — Testing and Validation**: Documents the verification and validation strategy employed to ensure system correctness, performance, and reliability. This chapter includes unit test coverage metrics, integration test scenarios for the detection-recognition-alert pipeline, performance benchmarks (FPS, alert latency, memory consumption), and edge-case testing for occlusion, lighting variation, and multi-target density.

- **Chapter 6 — Results and Discussion**: Presents empirical results from controlled testing environments and field trials, analyzing recognition accuracy rates, false positive/negative distributions, latency measurements across hardware configurations, and operator feedback from usability assessments.
- **Chapter 7 — Conclusion and Future Scope**: Summarizes the contributions of DrishtiX v3.0, evaluates the degree to which stated objectives have been met, and outlines the roadmap for future development — including multi-camera federation, cross-camera re-identification deployment, and potential integration with national databases for scaled law enforcement operations.

---